SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

CO2: Apply image enhancement and segmentation techniques to restore damaged photographs and identify meaningful image regions.

Assignment Details	Information	Assignment Details	Information
Student Name	Keshavaram L	Register Number	192511253
Department / Section	Computer science and Engineering	Academic Year / Semester	2025–2026
Faculty Name	Dr.P.Murali	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	05.11.2026	Date of Evaluation	

SCENARIO

A college archive has a collection of old family and institutional photographs that have become faded and damaged over time. The photographs show loss of contrast, yellow/sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading. Some images are slightly blurred, while others contain small torn or missing regions. The archive wants an Old Photograph Restoration System that (1) improves brightness, contrast and visual clarity while preserving original facial features and fine details, and (2) identifies damaged regions such as scratches, stains, dust spots, creases and torn areas so that they can be repaired or reviewed. The restored result must retain the historical appearance of the photograph and must not invent or alter important details.

You are the vision engineer. For every task, three candidate techniques are provided. Implement all three techniques on the same set of old photographs, compare their performance, select the most suitable technique, and justify the selection with experimental evidence. Pay special attention to preserving faces, clothing, background details and original photographic tone. State and justify any additional assumptions.

OPERATING DATA

Parameter	Operating Data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

OLD PHOTOGRAPH RESTORATION USING IMAGE PROCESSING TECHNIQUES

INTRODUCTION

Old photographs are valuable records of memories, family history, cultural heritage and historical events. However, photographs stored for many years may become damaged due to ageing, dust, moisture, scratches, stains, fading and improper storage. These problems reduce the visual quality of photographs and make important details difficult to identify.

Digital image processing provides different techniques to improve the appearance of old photographs. It uses computer algorithms to modify pixel values, enhance contrast, reduce noise, sharpen details, detect edges and analyse image regions.

This assignment implements ten image-processing tasks using Python and OpenCV. The implementation is performed in Google Colab using uploaded digital photographs. NumPy is used for numerical calculations, and Matplotlib is used to display the original and processed images.

The ten tasks covered in this assignment are:

1. Grey-level transformations.
2. Histogram enhancement.
3. Spatial filtering.
4. Image sharpening.
5. Frequency-domain low-pass filtering.
6. High-pass and homomorphic filtering.
7. Point, line and edge detection.
8. Image thresholding.
9. Image segmentation.
10. Colour enhancement and high-resolution strategies.

The main purpose is to understand how different image-processing techniques can improve the visual quality of old photographs.

TASK 1 – GREY-LEVEL TRANSFORMATIONS

AIM

To perform grey-level transformations on an old photograph using Python and OpenCV to improve image intensity and understand different brightness transformation techniques.

INTRODUCTION

Grey-level transformation is an important technique in digital image processing. It modifies the intensity values of pixels in a grayscale image to improve its visual appearance.

An image may contain dark regions, bright regions or low-contrast areas. Grey-level transformations help modify these intensity values to make certain details more visible.

A grayscale image contains pixel intensity values ranging from 0 to 255. Zero represents black, 255 represents white, and the values between them represent different shades of grey.

In this task, three transformations are implemented: image negative transformation, logarithmic transformation and gamma transformation.

THEORY

1. Image Negative Transformation

Image negative transformation reverses the intensity values of an image. Dark pixels become bright, and bright pixels become dark.

The mathematical formula is:

s = 255 - r

Where:

- r represents the original pixel intensity.
- s represents the transformed pixel intensity.
- 255 represents the maximum grayscale intensity.

For example, if the original pixel value is 50, the negative pixel value becomes 205.

This transformation is useful for observing details that may be difficult to identify in the original image.

2. Logarithmic Transformation

Logarithmic transformation is used to expand darker intensity values and compress higher intensity values.

The general formula is:

s = c log(1 + r)

Where:

- s represents the output pixel value.
- r represents the input pixel value.
- c represents a constant.
- log represents the logarithmic function.

This transformation can improve the visibility of details in darker regions of an image.

3. Gamma Transformation

Gamma transformation is a nonlinear intensity transformation used to adjust image brightness.

The formula is:

s = c(r^gamma)

Where:

- r represents the normalized input intensity.
- s represents the output intensity.
- c represents a constant.
- gamma represents the correction parameter.

When gamma is less than 1, the image generally becomes brighter. When gamma is greater than 1, the image generally becomes darker.

In this project, a gamma value of 0.5 is used.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the photograph into grayscale.

4. Apply negative transformation.
5. Apply logarithmic transformation.
6. Apply gamma transformation.
7. Display all transformed images.
8. Save the output images.
9. Stop the program.

PROCEDURE

First, the input photograph is uploaded into Google Colab. The image is read using the OpenCV function cv2.imread().

The photograph is converted into grayscale using cv2.cvtColor(). The negative transformation is performed by subtracting each pixel value from 255.

Next, logarithmic transformation is applied using NumPy mathematical functions. Gamma transformation is performed by raising the normalized pixel values to the power of 0.5.

Finally, the original grayscale image and the three transformed images are displayed using Matplotlib. The output images are saved in the project results folder.

OUTPUT

The following images are generated:

1. Original grayscale image.
2. Negative transformed image.
3. Logarithmic transformed image.
4. Gamma transformed image.

RESULT

Thus, grey-level transformations were successfully implemented using Python and OpenCV. Negative, logarithmic and gamma transformations produced different intensity effects and demonstrated how pixel values can be modified to improve image visibility.

TASK 2 – HISTOGRAM ENHANCEMENT

AIM

To perform histogram enhancement on an old photograph using histogram equalization and Contrast Limited Adaptive Histogram Equalization (CLAHE).

INTRODUCTION

Histogram enhancement is an image-processing technique used to improve the contrast of an image.

A histogram represents the distribution of pixel intensity values. In a grayscale image, the intensity values range from 0 to 255.

Old photographs may appear faded because their pixel intensities are concentrated within a limited range. Histogram enhancement helps redistribute these intensity values and improve the visual appearance.

This task implements histogram equalization and CLAHE using OpenCV.

THEORY

1. Image Histogram

An image histogram is a graphical representation of the frequency of different pixel intensity values.

The horizontal axis represents the intensity values from 0 to 255. The vertical axis represents the number of pixels having each intensity value.

A narrow histogram may indicate low contrast, while a wider distribution may indicate a greater range of intensity values.

2. Histogram Equalization

Histogram equalization is a contrast-enhancement technique that redistributes pixel intensity values using the cumulative distribution function.

The method attempts to make better use of the available intensity range.

The main steps are:

- Calculate the image histogram.
- Calculate the cumulative distribution.
- Normalize the cumulative distribution.
- Map the original pixel values to new intensity values.
- Generate the enhanced image.

OpenCV provides the function cv2.equalizeHist() to perform histogram equalization on grayscale images.

3. CLAHE

CLAHE stands for Contrast Limited Adaptive Histogram Equalization.

It is an improved form of adaptive histogram equalization. Instead of processing the complete image as a single region, CLAHE divides the image into smaller regions called tiles.

Histogram enhancement is performed separately within these regions. A contrast limit helps prevent excessive enhancement of noise.

In this project, the CLAHE parameters are:

- Clip limit: 2.0
- Tile grid size: 8 × 8

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the image into grayscale.
4. Calculate the original image histogram.
5. Apply histogram equalization.
6. Apply CLAHE.
7. Display the original and enhanced images.
8. Plot the original and enhanced histograms.
9. Save the output images.
10. Stop the program.

PROCEDURE

The old photograph is uploaded and converted into grayscale. Histogram equalization is applied using the OpenCV function cv2.equalizeHist().

Next, the CLAHE object is created using cv2.createCLAHE(). The clip limit is set to 2.0, and the tile grid size is set to 8 × 8.

The CLAHE operation is applied to the grayscale image. Both enhanced images are displayed along with the original image.

The histograms are plotted using Matplotlib to observe the distribution of pixel intensities before and after enhancement.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Histogram-equalized image.
3. CLAHE-enhanced image.
4. Original histogram.
5. Enhanced histogram.

RESULT

Thus, histogram enhancement was successfully implemented using Python and OpenCV. Histogram equalization and CLAHE demonstrated two different methods of improving image contrast. The output images and histograms can be compared to study the changes in intensity distribution.

TASK 3 – SPATIAL FILTERING

AIM

To perform spatial filtering on an old photograph using mean, Gaussian, median and bilateral filters to reduce unwanted noise.

INTRODUCTION

Spatial filtering is a digital image-processing technique in which operations are performed directly on image pixels.

A filter uses a small matrix called a kernel or mask. The kernel moves across the image and processes neighbouring pixels to calculate a new value for the central pixel.

Old photographs may contain unwanted noise due to ageing, scanning and poor image quality. Spatial filters can reduce certain types of noise and improve image smoothness.

This task implements four filtering techniques: mean filtering, Gaussian filtering, median filtering and bilateral filtering.

THEORY

1. Mean Filtering

Mean filtering is a smoothing technique that replaces each pixel with the average value of its neighbouring pixels.

A 5 × 5 kernel is used in this project.

The formula is:

Output pixel = Sum of neighbouring pixel values / Number of pixels

Mean filtering reduces small intensity variations and smooths the image. However, it may also blur edges and fine details.

2. Gaussian Filtering

Gaussian filtering is a smoothing technique that uses a Gaussian distribution to assign different weights to neighbouring pixels.

Pixels closer to the centre receive greater weights than pixels farther away.

Gaussian filtering reduces high-frequency noise and creates a smoother appearance.

The OpenCV function used is:

cv2.GaussianBlur()

3. Median Filtering

Median filtering replaces a pixel with the median value of the pixels in its neighbourhood.

Unlike mean filtering, it does not calculate the average. Instead, it sorts the neighbouring pixel values and selects the middle value.

Median filtering is particularly useful for reducing salt-and-pepper noise while preserving some image edges.

The OpenCV function used is:

cv2.medianBlur()

4. Bilateral Filtering

Bilateral filtering is an edge-preserving smoothing technique.

It considers both the spatial distance between pixels and the difference in their intensity values.

Pixels that are close together and have similar intensity values receive greater influence during filtering.

Bilateral filtering can smooth image regions while preserving important boundaries.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the image into grayscale.
4. Apply mean filtering.
5. Apply Gaussian filtering.
6. Apply median filtering.
7. Apply bilateral filtering.
8. Display all filtered images.
9. Save the output images.
10. Stop the program.

PROCEDURE

The photograph is uploaded into Google Colab and converted into grayscale.

Mean filtering is performed using cv2.blur() with a 5 × 5 kernel.

Gaussian filtering is performed using cv2.GaussianBlur() with a 5 × 5 kernel.

Median filtering is performed using cv2.medianBlur() with a kernel size of 5.

Bilateral filtering is performed using cv2.bilateralFilter() with the parameters 9, 75 and 75.

The original image and all filtered images are displayed for comparison.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Mean-filtered image.
3. Gaussian-filtered image.
4. Median-filtered image.
5. Bilateral-filtered image.

RESULT

Thus, spatial filtering was successfully implemented using Python and OpenCV. Four filtering techniques were applied to the input photograph. Their outputs demonstrate different approaches to smoothing and noise reduction.

TASK 4 – IMAGE SHARPENING

AIM

To perform image sharpening using a sharpening kernel and unsharp masking to improve the visibility of edges and details in an old photograph.

INTRODUCTION

Image sharpening is a technique used to enhance the edges and fine details of an image.

Old photographs may appear blurred due to ageing, poor scanning, camera focus problems or low image quality.

Sharpening methods increase the visibility of intensity transitions and make certain image structures appear clearer.

This task implements kernel-based sharpening and unsharp masking.

THEORY

1. Kernel-Based Sharpening

A sharpening kernel is a small matrix used to enhance image edges.

The kernel used in this project is:

[0 -1 0]

[-1 5 -1]

[0 -1 0]

The centre value is positive, while the surrounding values are negative.

The filter emphasizes differences between the centre pixel and its neighbouring pixels. This helps enhance edges and local details.

The OpenCV function cv2.filter2D() is used to apply the sharpening kernel.

2. Unsharp Masking

Unsharp masking is a sharpening technique that combines the original image with a blurred version of the image.

First, the image is blurred using Gaussian filtering. The blurred image is then subtracted from the original image to obtain high-frequency details.

The sharpened image is produced by adding these details back to the original image.

The general formula is:

Sharpened Image = Original Image + k × (Original Image - Blurred Image)

Where k controls the sharpening strength.

In this project, the original image is multiplied by 1.5, and the blurred image is multiplied by -0.5.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the photograph into grayscale.
4. Define the sharpening kernel.
5. Apply the sharpening kernel.
6. Apply Gaussian blur to the original image.
7. Perform unsharp masking.
8. Display the original and sharpened images.
9. Save the results.
10. Stop the program.

PROCEDURE

The input photograph is uploaded and converted into grayscale.

A sharpening kernel is defined using NumPy. The kernel is applied using the OpenCV function cv2.filter2D().

Next, Gaussian blur is applied to the original grayscale image using cv2.GaussianBlur().

Unsharp masking is performed using cv2.addWeighted(). The original image and blurred image are combined using the weights 1.5 and -0.5.

The original image, kernel-sharpened image and unsharp-masked image are displayed and saved.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Kernel-sharpened image.
3. Unsharp-masked image.

RESULT

Thus, image sharpening was successfully implemented using Python and OpenCV. Kernel-based sharpening and unsharp masking were applied to enhance the appearance of edges and details in the photograph.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING

AIM

To perform frequency-domain low-pass filtering on an old photograph using the two-dimensional Fourier transform.

INTRODUCTION

Frequency-domain image processing is a technique in which an image is converted from the spatial domain into the frequency domain for processing.

An image contains different spatial frequencies. Low-frequency components represent slowly changing intensity regions, while high-frequency components represent rapid intensity changes such as edges and fine details.

Low-pass filtering allows low-frequency components to pass and reduces high-frequency components.

This method can be used to smooth images and reduce certain types of high-frequency noise.

THEORY

1. Fourier Transform

The Fourier transform converts an image from the spatial domain into the frequency domain.

For a two-dimensional image, the Fourier transform is represented as:

$F(u,v) = \Sigma\Sigma\ f(x,y)e^{[-j2\pi(ux/M + vy/N)]}$

Where:

- f(x,y) represents the original image.
- F(u,v) represents the frequency-domain image.
- x and y represent spatial coordinates.
- u and v represent frequency coordinates.
- M and N represent image dimensions.

2. Low-Pass Filtering

A low-pass filter allows low-frequency components to pass and removes or reduces higher-frequency components.

The frequency-domain filtering operation is:

$G(u,v) = H(u,v)F(u,v)$

Where:

- F(u,v) is the Fourier transform of the original image.
- H(u,v) is the filter mask.
- G(u,v) is the filtered frequency representation.

In this project, a circular low-pass mask is created around the centre of the shifted Fourier transform.

The radius is selected based on the image dimensions.

3. Inverse Fourier Transform

After filtering, the inverse Fourier transform converts the frequency-domain result back into the spatial domain.

This produces the filtered image.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the photograph into grayscale.
4. Calculate the two-dimensional Fourier transform.
5. Shift the zero-frequency component to the centre.
6. Create a circular low-pass mask.
7. Multiply the frequency representation by the mask.
8. Apply the inverse shift.

9. Calculate the inverse Fourier transform.
10. Normalize the output and display it.
11. Save the filtered image.
12. Stop the program.

PROCEDURE

The photograph is uploaded and converted into grayscale.

The Fourier transform is calculated using np.fft.fft2(). The zero-frequency component is shifted to the centre using np.fft.fftshift().

A circular mask is created using OpenCV. The mask allows frequencies inside the selected radius to pass while reducing frequencies outside the radius.

The shifted Fourier transform is multiplied by the mask. The inverse shift is then applied using np.fft.ifftshift().

Finally, the inverse Fourier transform is calculated using np.fft.ifft2(). The absolute values are converted into an 8-bit image and displayed.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Frequency-domain low-pass filtered image.

RESULT

Thus, frequency-domain low-pass filtering was successfully implemented using NumPy and OpenCV. The Fourier transform was used to process image frequencies, and the inverse Fourier transform generated the filtered image.

TASK 6 – HIGH-PASS AND HOMOMORPHIC FILTERING

AIM

To implement high-pass filtering and homomorphic filtering on an old photograph using frequency-domain image-processing techniques.

INTRODUCTION

High-pass filtering and homomorphic filtering are frequency-domain processing techniques used to modify different characteristics of an image.

High-pass filtering emphasizes rapid intensity changes and highlights edges and fine structures.

Homomorphic filtering is used to improve image appearance by separating illumination-related variations from reflectance-related information.

These techniques demonstrate how frequency-domain processing can be used to analyse and enhance old photographs.

THEORY

1. High-Pass Filtering

High-pass filtering allows higher-frequency components to pass and reduces lower-frequency components.

High-frequency components represent rapid changes in pixel intensity. These changes commonly occur around edges and fine details.

The filtering operation is:

$G(u,v) = H(u,v)F(u,v)$

In this project, a circular mask is created with zero values near the centre and one values outside the selected radius.

The central low-frequency region is removed, while higher-frequency components are retained.

The inverse Fourier transform is then used to generate the high-pass filtered image.

2. Homomorphic Filtering

Homomorphic filtering is used to improve an image affected by non-uniform illumination.

An image can be represented as the product of illumination and reflectance:

$f(x,y) = i(x,y) \times r(x,y)$

Where:

- $i(x,y)$ represents illumination.
- $r(x,y)$ represents reflectance.
- $f(x,y)$ represents the observed image.

The logarithmic transformation converts the multiplication into addition:

$\ln f(x,y) = \ln i(x,y) + \ln r(x,y)$

The transformed image is processed in the frequency domain using a homomorphic filter.

The filter can reduce slowly changing illumination components and emphasize reflectance-related details.

In this project, the filter uses the following parameters:

- Gamma low = 0.5
- Gamma high = 1.5
- c = 1.0
- D0 = one-eighth of the smaller image dimension

After filtering, the inverse Fourier transform is applied and the exponential transformation is used to obtain the output image.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the image into grayscale.
4. Calculate the Fourier transform.
5. Create a high-pass filter mask.
6. Apply high-pass filtering.
7. Calculate the inverse Fourier transform.
8. Apply logarithmic transformation to the image.
9. Calculate the Fourier transform of the logarithmic image.
10. Construct the homomorphic filter.
11. Apply the filter in the frequency domain.
12. Apply the inverse Fourier transform.
13. Perform exponential transformation.

14. Display and save both filtered images.
15. Stop the program.

PROCEDURE

The input photograph is uploaded and converted into grayscale.

For high-pass filtering, the Fourier transform is calculated using NumPy. The frequency representation is shifted to the centre, and a circular high-pass mask is created.

The frequency representation is multiplied by the mask. The inverse Fourier transform is applied to obtain the high-pass filtered image.

For homomorphic filtering, the grayscale image is converted into floating-point values and logarithmic transformation is applied.

The Fourier transform of the logarithmic image is calculated. A homomorphic filter is created using the distance from the frequency centre.

The filtered frequency representation is converted back into the spatial domain. Exponential transformation is then applied, and the final image is normalized.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. High-pass filtered image.
3. Homomorphic filtered image.

RESULT

Thus, high-pass filtering and homomorphic filtering were successfully implemented. The techniques demonstrated how frequency-domain operations can emphasize high-frequency image details and modify illumination-related variations.

TASK 7 – POINT, LINE AND EDGE DETECTION

AIM

To perform point detection, line detection and edge detection on an old photograph using image-processing filters and the Canny edge detection algorithm.

INTRODUCTION

Point, line and edge detection are important techniques in digital image processing. They help identify significant structures and boundaries within an image.

Old photographs may contain important objects, shapes and boundaries that are difficult to observe due to fading or low contrast.

Point detection identifies isolated intensity changes. Line detection identifies linear structures. Edge detection identifies boundaries between different image regions.

This task implements point detection, horizontal line detection, vertical line detection and Canny edge detection.

THEORY

1. Point Detection

Point detection identifies isolated pixels that differ significantly from their surrounding pixels.

A point detection kernel is used:

[-1 -1 -1]

[-1 8 -1]

[-1 -1 -1]

The filter responds strongly when the centre pixel differs from its neighbours.

The absolute filter response is converted into an 8-bit image for display.

2. Line Detection

Line detection identifies linear structures in an image.

Different kernels can be designed to detect horizontal, vertical and diagonal lines.

In this project, a horizontal line detection kernel and a vertical line detection kernel are used.

Horizontal line detection identifies intensity changes associated with horizontal structures.

Vertical line detection identifies intensity changes associated with vertical structures.

3. Edge Detection

An edge represents a boundary where image intensity changes rapidly.

Canny edge detection is a widely used method for detecting image edges.

The major stages of Canny edge detection are:

1. Noise reduction using Gaussian filtering.
2. Gradient calculation.
3. Non-maximum suppression.
4. Double thresholding.
5. Edge tracking by hysteresis.

In this project, Canny edge detection is performed using thresholds of 50 and 150.

The OpenCV function used is:

cv2.Canny(gray, 50, 150)

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the image into grayscale.
4. Define the point detection kernel.
5. Apply point detection.
6. Define the horizontal line kernel.
7. Apply horizontal line detection.
8. Define the vertical line kernel.
9. Apply vertical line detection.
10. Apply Canny edge detection.
11. Display the results.

12. Save all output images.
13. Stop the program.

PROCEDURE

The old photograph is uploaded and converted into grayscale.

A point detection kernel is created using NumPy. The kernel is applied using cv2.filter2D().

Next, a horizontal line detection kernel is defined and applied to the grayscale image. A vertical line detection kernel is also applied.

Canny edge detection is performed using cv2.Canny() with lower and upper thresholds of 50 and 150.

All output images are displayed together for comparison and saved in the results folder.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Point detection image.
3. Horizontal line detection image.
4. Vertical line detection image.
5. Canny edge detection image.

RESULT

Thus, point detection, line detection and edge detection were successfully implemented using Python and OpenCV. The output images demonstrate how different filters identify important image structures and boundaries.

TASK 8 – IMAGE THRESHOLDING

AIM

To perform image thresholding using binary, inverse, adaptive and Otsu thresholding methods on an old photograph.

INTRODUCTION

Image thresholding is a segmentation technique used to convert a grayscale image into a binary image.

A binary image generally contains two intensity values: 0 and 255. Zero represents black, and 255 represents white.

Thresholding separates pixels into different groups based on their intensity values.

It is useful for identifying regions, objects and boundaries in an image.

This task implements four thresholding techniques.

THEORY

1. Binary Thresholding

Binary thresholding compares every pixel with a specified threshold value.

The mathematical representation is:

$g(x,y) = 255$, if $f(x,y) > T$

$g(x,y) = 0$, otherwise

Where T represents the threshold value.

In this project, the threshold value is 127.

Pixels greater than 127 become white, and other pixels become black.

2. Inverse Binary Thresholding

Inverse binary thresholding performs the opposite operation.

Pixels greater than the threshold become black, while pixels less than or equal to the threshold become white.

This method produces an inverted binary representation.

3. Adaptive Thresholding

Adaptive thresholding calculates different threshold values for different image regions.

It is useful when an image contains uneven lighting or different brightness levels.

In this project, Gaussian adaptive thresholding is used with a block size of 11 and a constant value of 2.

4. Otsu Thresholding

Otsu thresholding automatically calculates a threshold value based on the grayscale histogram.

It attempts to separate two intensity classes by minimizing the within-class variance.

This method is useful when the image contains distinguishable foreground and background intensity distributions.

OpenCV provides the function cv2.threshold() with the THRESH_OTSU flag.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the image into grayscale.
4. Apply binary thresholding with threshold 127.
5. Apply inverse binary thresholding.
6. Apply adaptive Gaussian thresholding.
7. Apply Otsu thresholding.
8. Display all thresholded images.
9. Save the output images.
10. Stop the program.

PROCEDURE

The input photograph is uploaded and converted into grayscale.

Binary thresholding is performed using cv2.threshold() with a threshold value of 127.

Inverse thresholding is performed using cv2.THRESH_BINARY_INV.

Adaptive Gaussian thresholding is performed using cv2.adaptiveThreshold().

Finally, Otsu thresholding is performed by setting the threshold value to zero and enabling cv2.THRESH_OTSU.

All four output images are displayed and saved.

OUTPUT

The following outputs are generated:

1. Original grayscale image.
2. Binary thresholded image.

3. Inverse thresholded image.
4. Adaptive thresholded image.
5. Otsu thresholded image.

RESULT

Thus, image thresholding was successfully implemented using Python and OpenCV. Binary, inverse, adaptive and Otsu thresholding methods were applied to understand different approaches to separating image regions based on pixel intensity.

TASK 9 – IMAGE SEGMENTATION

AIM

To perform image segmentation using region growing, split-based processing and edge linking techniques on an old photograph.

INTRODUCTION

Image segmentation is the process of dividing an image into meaningful regions or groups of pixels.

It is an important operation in digital image processing because it helps identify objects, boundaries and areas with similar characteristics.

In old photographs, segmentation can help analyse different regions and identify areas that may require further examination.

This task implements three approaches:

1. Region growing.
2. Split-based processing.
3. Edge linking.

The program also supports optional evaluation using a manually prepared ground-truth mask.

THEORY

1. Region Growing

Region growing is a segmentation method that starts from a selected pixel called the seed point.

The algorithm examines neighbouring pixels and adds them to the region when their intensity values satisfy a similarity condition.

In this project, four-connected neighbourhoods are used.

The intensity difference between a neighbouring pixel and the original seed intensity must be less than or equal to 18.

The steps are:

- Select a seed pixel.
- Add the seed pixel to a queue.
- Examine its four neighbouring pixels.
- Compare their intensities with the seed intensity.
- Add suitable pixels to the segmented region.
- Continue until no more suitable neighbouring pixels remain.

The seed location affects the segmentation result. The current program uses the centre of the image as the default seed.

For meaningful analysis, a seed should be selected inside the region being investigated.

2. Split-Based Processing

Split-based processing is a region-based image analysis technique.

The image is divided into smaller rectangular regions. Each region is examined using a homogeneity condition.

In this project, the standard deviation of pixel intensities is used to examine the variation within each region.

If the standard deviation is less than 15, or the region becomes too small, the region is retained. Otherwise, it is divided into four smaller regions.

The process continues recursively.

The program compares the original grayscale image with the resulting split-based image and calculates an absolute difference.

A threshold of 18 is applied to produce a residual mask.

This implementation demonstrates recursive splitting and residual generation. It is not a complete split-and-merge algorithm because it does not perform a separate region-merging stage.

3. Edge Linking

Edge linking is used to connect nearby edge pixels and form more continuous boundaries.

First, Canny edge detection is applied to the grayscale image.

Then, morphological closing is performed using an elliptical structuring element of size 5 × 5.

Morphological closing helps connect small gaps between nearby edge regions.

Contours are then detected using cv2.findContours().

The detected contours are drawn to produce a boundary image.

4. Dice and IoU Evaluation

When a manually prepared ground-truth mask is available, segmentation results can be evaluated using Dice coefficient and Intersection over Union.

Dice coefficient:

Dice = 2 × Intersection / (Predicted pixels + Ground-truth pixels)

Intersection over Union:

IoU = Intersection / Union

A higher score generally indicates greater overlap between the predicted region and the ground-truth region.

These metrics are calculated only when an annotated ground-truth mask is supplied.

ALGORITHM

1. Start the program.
2. Read the input photograph.
3. Convert the photograph into grayscale.
4. Select a seed point for region growing.
5. Perform region growing using an intensity threshold of 18.
6. Divide the image recursively using split-based processing.

7. Calculate the residual difference image.
8. Apply thresholding to generate the residual mask.
9. Detect edges using Canny edge detection.
10. Apply morphological closing.
11. Detect contours and generate boundary images.
12. Display all segmentation outputs.
13. If a ground-truth mask is supplied, calculate Dice and IoU.
14. Save the output images and evaluation values.
15. Stop the program.

PROCEDURE

The input photograph is uploaded and converted into grayscale.

The region-growing algorithm starts from the centre of the image by default. A queue is used to examine neighbouring pixels. Pixels satisfying the intensity similarity condition are included in the segmented region.

Next, split-based processing divides the image into smaller regions. The standard deviation is calculated for each region. Regions satisfying the homogeneity condition are retained, while other regions are divided recursively.

The residual image is calculated using the absolute difference between the original grayscale image and the split-based result. Thresholding is applied to produce the residual mask.

For edge linking, Canny edge detection is performed using thresholds of 50 and 130. Morphological closing is then applied using a 5×5 elliptical kernel.

Contours are detected and drawn to create the boundary output.

If the user has a manually annotated ground-truth mask, it can be uploaded when the program asks:

"Upload a ground-truth damage mask for Task 9? (y/n):"

If no ground-truth mask is available, the user can enter n. The program continues without calculating Dice and IoU.

OUTPUT

The following outputs are generated:

1. Region-growing result.
2. Split-merge residual mask.
3. Split-based visual result.
4. Edge-linked image.
5. Edge boundary image.
6. Optional Dice and IoU evaluation CSV file.

RESULT

Thus, image segmentation was successfully demonstrated using region growing, recursive split-based processing and edge linking. The program generated different segmentation and boundary outputs. Optional evaluation was also implemented using Dice coefficient and IoU when an annotated ground-truth mask was available.

TASK 10 – COLOUR ENHANCEMENT AND HIGH-RESOLUTION STRATEGIES

AIM

To implement colour enhancement, image resizing and candidate damage-region restoration using luminance enhancement and inpainting techniques.

INTRODUCTION

Old photographs may lose their original appearance due to fading, poor contrast, uneven illumination and physical damage.

Colour enhancement improves the appearance of photographs by adjusting image brightness and contrast. Image resizing changes the dimensions of an image while attempting to maintain useful visual information.

Inpainting is a technique used to estimate missing or damaged image regions using information from nearby pixels.

This task implements three strategies:

1. Retaining the original image dimensions while enhancing luminance.
2. Resizing large images and enhancing luminance.
3. Applying candidate damage detection and inpainting.

THEORY

1. LAB Colour Space

The LAB colour space represents an image using three channels:

- L represents lightness.
- A represents the green-to-red colour component.
- B represents the blue-to-yellow colour component.

The L channel contains luminance information, while the A and B channels contain colour information.

Enhancing the L channel separately allows brightness and contrast to be adjusted while retaining the original colour channels.

In this project, the BGR image is converted into LAB colour space using:

cv2.cvtColor(im, cv2.COLOR_BGR2LAB)

The L, A and B channels are separated using cv2.split().

CLAHE is applied to the L channel, and the channels are combined again to produce the enhanced colour image.

2. Image Resizing

Image resizing changes the width and height of an image.

Large photographs may require more memory and processing time. Resizing can make processing easier.

In this project, the maximum image dimension is limited to approximately 2000 pixels.

The scaling factor is calculated as:

scale = min(1.0, 2000.0 / max(height, width))

If the original image is larger than the selected limit, it is resized using cv2.resize() with area interpolation.

If the original image is already within the limit, a copy of the image is used.

3. Candidate Damage Detection

Damage detection attempts to identify regions that may contain scratches, marks or unwanted variations.

In this project, a median-filtered image is compared with the original grayscale image.

The median-filtered image represents local intensity information. The absolute difference is calculated between the original image and the filtered image.

A threshold of 28 is applied to create a candidate damage mask.

Morphological opening is then performed using a 3 × 3 elliptical kernel to remove some small isolated regions.

This mask is only a heuristic candidate mask. It may also select genuine image textures, edges, hair and other important details.

Therefore, it must be visually inspected before accepting the restoration result.

4. Image Inpainting

Image inpainting estimates pixel values inside a selected mask.

The surrounding image information is used to estimate replacement values.

In this project, OpenCV's Telea inpainting method is used:

cv2.inpaint(corrected, mask, 3, cv2.INPAINT_TELEA)

The inpainting radius is set to 3.

Inpainting can help fill small selected regions, but it cannot recover the exact original information that has been completely lost.

ALGORITHM

1. Start the program.
2. Read the input colour photograph.
3. Obtain the image dimensions.
4. Convert the image into LAB colour space.
5. Separate the L, A and B channels.
6. Apply CLAHE to the L channel.
7. Recombine the channels to produce the first enhanced image.
8. Calculate the resizing factor.
9. Resize the image if its dimensions exceed the selected limit.
10. Convert the resized image into LAB colour space.
11. Apply CLAHE to the resized image's L channel.
12. Convert the enhanced image back to BGR.
13. Convert the original image into grayscale.
14. Apply median filtering.
15. Calculate the absolute difference.
16. Generate a candidate damage mask using thresholding.
17. Apply morphological opening.
18. Apply Telea inpainting to the enhanced image.
19. Display all outputs.
20. Save the output images.

21. Stop the program.

PROCEDURE

The input photograph is uploaded and its height and width are obtained.

The image is converted from BGR to LAB colour space. The three channels are separated, and CLAHE is applied to the L channel.

The enhanced L channel is combined with the original A and B channels. This produces the first colour-enhanced image while retaining the original image dimensions.

Next, the resizing factor is calculated. If the largest image dimension exceeds 2000 pixels, the image is resized using area interpolation.

The resized image is converted into LAB colour space. CLAHE is applied to its luminance channel, and the image is converted back into BGR format.

For candidate damage detection, the original photograph is converted into grayscale. A median filter with a kernel size of 9 is applied.

The absolute difference between the original grayscale image and the median-filtered image is calculated. A threshold of 28 is applied to generate a candidate damage mask.

Morphological opening is performed using a 3 × 3 elliptical kernel.

Finally, the corrected colour image and candidate damage mask are supplied to the Telea inpainting algorithm with an inpainting radius of 3.

The original image, enhanced image, resized image, inpainted image and candidate damage mask are displayed and saved.

OUTPUT

The following outputs are generated:

1. Original colour photograph.
2. A – Retained-size luminance-enhanced image.
3. B – Resized luminance-enhanced image.
4. C – Inpainted image.
5. Candidate damage mask.

RESULT

Thus, colour enhancement, image resizing and inpainting were successfully implemented using Python and OpenCV. CLAHE was applied to the luminance channel, and a heuristic mask was generated to demonstrate damage-region estimation. The Telea inpainting method was used to estimate replacement values in selected regions.

IMPLEMENTATION PART

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time
import csv
import shutil
from pathlib import Path
from collections import deque

# ============================================================
# OLD PHOTOGRAPH RESTORATION
# Complete implementation: Task 1 to Task 10
# ============================================================

try:
    from google.colab import files
    COLAB = True
except ImportError:
    COLAB = False

OUT = Path("old_photo_restoration_results")
OUT.mkdir(exist_ok=True)

FORMATS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}

# ============================================================
# COMMON HELPER FUNCTIONS
# ============================================================

def load_image(path):
    image = cv2.imread(str(path))

    if image is None:
        raise ValueError("Unable to read image: " + str(path))
```

```
    return image

def to_gray(im):
    if len(im.shape) == 2:
        return im.copy()

    return cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)

def save(filename, image):
    path = OUT / filename
    cv2.imwrite(str(path), image)

def write_csv(filename, rows):
    if not rows:
        return

    path = OUT / filename

    with open(path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)

def show(images, title="Results", columns=3):
    count = len(images)
    rows = int(np.ceil(count / columns))

    plt.figure(figsize=(15, 4 * rows))

    for i, (name, image) in enumerate(images.items() if isinstance(images, dict) else images):
        plt.subplot(rows, columns, i + 1)

        if len(image.shape) == 2:
            plt.imshow(image, cmap="gray")
        else:
```

```
            plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

        plt.title(name)
        plt.axis("off")

    plt.suptitle(title)
    plt.tight_layout()
    plt.show()

def timed(function, *args):
    start = time.time()
    result = function(*args)
    seconds = time.time() - start
    return result, seconds

# ============================================================
# TASK 1: GREY-LEVEL TRANSFORMATIONS
# ============================================================

def task1(im, stem):
    gray = to_gray(im)

    negative = 255 - gray

    log_image = np.uint8(
        255 * np.log1p(gray.astype(np.float32)) /
        np.log(256)
    )

    gamma = np.uint8(
        255 * np.power(gray / 255.0, 0.5)
    )

    outputs = {
        "original_gray": gray,
        "negative": negative,
```

```
        "log_transformation": log_image,
        "gamma_transformation": gamma
    }

    for name, image in outputs.items():
        save(f"{stem}_T01_{name}.png", image)

    show(outputs, f"{stem} - Task 1")

    return outputs

# ============================================================
# TASK 2: HISTOGRAM ENHANCEMENT
# ============================================================

def task2(im, stem):
    gray = to_gray(im)

    equalized = cv2.equalizeHist(gray)

    clahe = cv2.createCLAHE(
        clipLimit=2.0,
        tileGridSize=(8, 8)
    )

    clahe_image = clahe.apply(gray)

    outputs = {
        "original": gray,
        "histogram_equalization": equalized,
        "CLAHE": clahe_image
    }

    for name, image in outputs.items():
        save(f"{stem}_T02_{name}.png", image)
```

```
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.hist(gray.ravel(), 256, [0, 256])
    plt.title("Original Histogram")

    plt.subplot(1, 2, 2)
    plt.hist(clahe_image.ravel(), 256, [0, 256])
    plt.title("CLAHE Histogram")

    plt.tight_layout()
    plt.savefig(OUT / f"{stem}_T02_histograms.png")
    plt.show()

    show(outputs, f"{stem} - Task 2")

    return outputs

# ============================================================
# TASK 3: SPATIAL FILTERING
# ============================================================

def task3(im, stem):
    gray = to_gray(im)

    mean_filter = cv2.blur(gray, (5, 5))

    gaussian_filter = cv2.GaussianBlur(
        gray, (5, 5), 0
    )

    median_filter = cv2.medianBlur(gray, 5)

    bilateral_filter = cv2.bilateralFilter(
        gray, 9, 75, 75
    )
```

```
    outputs = {
        "original": gray,
        "mean_filter": mean_filter,
        "gaussian_filter": gaussian_filter,
        "median_filter": median_filter,
        "bilateral_filter": bilateral_filter
    }

    for name, image in outputs.items():
        save(f"{stem}_T03_{name}.png", image)

    show(outputs, f"{stem} - Task 3")

    return outputs

# ============================================================
# TASK 4: IMAGE SHARPENING
# ============================================================

def task4(im, stem):
    gray = to_gray(im)

    kernel = np.array([
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0]
    ])

    sharpened = cv2.filter2D(
        gray, -1, kernel
    )

    blurred = cv2.GaussianBlur(
        gray, (5, 5), 0
    )
```

```
    unsharp = cv2.addWeighted(
        gray, 1.5, blurred, -0.5, 0
    )

    outputs = {
        "original": gray,
        "kernel_sharpening": sharpened,
        "unsharp_masking": unsharp
    }

    for name, image in outputs.items():
        save(f"{stem}_T04_{name}.png", image)

    show(outputs, f"{stem} - Task 4")

    return outputs

# ============================================================
# TASK 5: FREQUENCY DOMAIN LOW-PASS FILTERING
# ============================================================

def task5(im, stem):
    gray = to_gray(im)

    rows, cols = gray.shape

    dft = np.fft.fft2(gray)
    shifted = np.fft.fftshift(dft)

    crow = rows // 2
    ccol = cols // 2

    mask = np.zeros((rows, cols), np.float32)

    radius = max(1, min(rows, cols) // 8)
```

```
    cv2.circle(
        mask,
        (ccol, crow),
        radius,
        1,
        -1
    )

    filtered = shifted * mask

    inverse_shift = np.fft.ifftshift(filtered)

    result = np.fft.ifft2(inverse_shift)

    result = np.abs(result)

    result = np.uint8(
        np.clip(result, 0, 255)
    )

    outputs = {
        "original": gray,
        "low_pass_filtered": result
    }

    save(f"{stem}_T05_low_pass.png", result)

    show(outputs, f"{stem} - Task 5")

    return outputs

# ============================================================
# TASK 6: HIGH-PASS AND HOMOMORPHIC FILTERING
# ============================================================
```

```
def task6(im, stem):
    gray = to_gray(im)

    rows, cols = gray.shape

    dft = np.fft.fft2(gray)
    shifted = np.fft.fftshift(dft)

    crow = rows // 2
    ccol = cols // 2

    mask = np.ones((rows, cols), np.float32)

    radius = max(1, min(rows, cols) // 8)

    cv2.circle(
        mask,
        (ccol, crow),
        radius,
        0,
        -1
    )

    high_filtered = shifted * mask

    inverse = np.fft.ifft2(
        np.fft.ifftshift(high_filtered)
    )

    high_pass = np.abs(inverse)

    high_pass = np.uint8(
        cv2.normalize(
            high_pass,
            None,
            0,
```

```
        255,
        cv2.NORM_MINMAX
    )
)

# Homomorphic filtering

gray_float = gray.astype(np.float32) + 1

log_image = np.log(gray_float)

dft_log = np.fft.fft2(log_image)
shifted_log = np.fft.fftshift(dft_log)

y, x = np.ogrid[:rows, :cols]

distance = np.sqrt(
    (x - ccol) ** 2 +
    (y - crow) ** 2
)

D0 = max(1, min(rows, cols) // 8)

gamma_low = 0.5
gamma_high = 1.5
c = 1.0

H = (
    gamma_high - gamma_low
) * (
    1 - np.exp(
        -c * (distance ** 2) / (D0 ** 2)
    )
) + gamma_low

homomorphic_filtered = shifted_log * H
```

```
    inverse_log = np.fft.ifft2(
        np.fft.ifftshift(homomorphic_filtered)
    )

    exponential = np.exp(np.real(inverse_log))

    homomorphic = np.uint8(
        cv2.normalize(
            exponential,
            None,
            0,
            255,
            cv2.NORM_MINMAX
        )
    )

    outputs = {
        "original": gray,
        "high_pass": high_pass,
        "homomorphic": homomorphic
    }

    for name, image in outputs.items():
        save(f"{stem}_T06_{name}.png", image)

    show(outputs, f"{stem} - Task 6")

    return outputs

# ============================================================
# TASK 7: POINT, LINE AND EDGE DETECTION
# ============================================================

def task7(im, stem):
    gray = to_gray(im)
```

```
# Point detection
point_kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
])

point_response = cv2.filter2D(
    gray, cv2.CV_32F, point_kernel
)

point_image = np.uint8(
    np.clip(np.abs(point_response), 0, 255)
)

# Horizontal line detection
horizontal_kernel = np.array([
    [-1, -1, -1],
    [2, 2, 2],
    [-1, -1, -1]
])

horizontal = cv2.filter2D(
    gray, -1, horizontal_kernel
)

# Vertical line detection
vertical_kernel = np.array([
    [-1, 2, -1],
    [-1, 2, -1],
    [-1, 2, -1]
])

vertical = cv2.filter2D(
    gray, -1, vertical_kernel
```

```
    )

    # Edge detection
    canny = cv2.Canny(gray, 50, 150)

    outputs = {
        "original": gray,
        "point_detection": point_image,
        "horizontal_lines": horizontal,
        "vertical_lines": vertical,
        "canny_edges": canny
    }

    for name, image in outputs.items():
        save(f"{stem}_T07_{name}.png", image)

    show(outputs, f"{stem} - Task 7")

    return outputs

# ============================================================
# TASK 8: IMAGE THRESHOLDING
# ============================================================

def task8(im, stem):
    gray = to_gray(im)

    _, binary = cv2.threshold(
        gray,
        127,
        255,
        cv2.THRESH_BINARY
    )

    _, inverse = cv2.threshold(
        gray,
```

```
    127,
    255,
    cv2.THRESH_BINARY_INV
)

adaptive = cv2.adaptiveThreshold(
    gray,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

_, otsu = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

outputs = {
    "original": gray,
    "binary_threshold": binary,
    "inverse_threshold": inverse,
    "adaptive_threshold": adaptive,
    "otsu_threshold": otsu
}

for name, image in outputs.items():
    save(f"{stem}_T08_{name}.png", image)

show(outputs, f"{stem} - Task 8")

return outputs
```

```
# ============================================================
# TASK 9: IMAGE SEGMENTATION
# ============================================================

def region_growing(im, seed, threshold=18):
    gray = to_gray(im)

    h, w = gray.shape

    sy, sx = seed

    if not (0 <= sy < h and 0 <= sx < w):
        raise ValueError("Region-growing seed is outside the image.")

    visited = np.zeros((h, w), np.uint8)
    result = np.zeros((h, w), np.uint8)

    seed_value = int(gray[sy, sx])

    queue = deque()
    queue.append((sy, sx))

    visited[sy, sx] = 1

    # Limit processed pixels to avoid excessive runtime.
    max_pixels = min(h * w, 1000000)
    processed = 0

    while queue and processed < max_pixels:
        y, x = queue.popleft()
        processed += 1

        value = int(gray[y, x])

        if abs(value - seed_value) <= threshold:
            result[y, x] = 255
```

```
            for dy, dx in [
                (-1, 0),
                (1, 0),
                (0, -1),
                (0, 1)
            ]:
                ny = y + dy
                nx = x + dx

                if (
                    0 <= ny < h and
                    0 <= nx < w and
                    visited[ny, nx] == 0
                ):
                    visited[ny, nx] = 1
                    queue.append((ny, nx))

    return result

def split_merge_demo(im):
    gray = to_gray(im)

    result = np.zeros_like(gray)

    def split(x, y, w, h):
        block = gray[y:y+h, x:x+w]

        if block.size == 0:
            return

        if (
            np.std(block) < 15 or
            w <= 1 or
            h <= 1
        ):
```

```
            result[y:y+h, x:x+w] = block
            return

        w2 = w // 2
        h2 = h // 2

        if w2 == 0 or h2 == 0:
            result[y:y+h, x:x+w] = block
            return

        split(x, y, w2, h2)
        split(x+w2, y, w-w2, h2)
        split(x, y+h2, w2, h-h2)
        split(x+w2, y+h2, w-w2, h-h2)

    split(0, 0, gray.shape[1], gray.shape[0])

    residual = cv2.absdiff(gray, result)

    mask = cv2.threshold(
        residual,
        18,
        255,
        cv2.THRESH_BINARY
    )[1]

    return mask, result

def edge_linking(im):
    gray = to_gray(im)

    edges = cv2.Canny(gray, 50, 130)

    kernel = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE,
        (5, 5)
```

```
    )

    linked = cv2.morphologyEx(
        edges,
        cv2.MORPH_CLOSE,
        kernel
    )

    contours, _ = cv2.findContours(
        linked,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    boundary = np.zeros_like(gray)

    cv2.drawContours(
        boundary,
        contours,
        -1,
        255,
        1
    )

    return linked, boundary

def dice_iou(predicted, ground_truth):
    predicted = predicted > 0
    ground_truth = ground_truth > 0

    intersection = np.logical_and(
        predicted,
        ground_truth
    ).sum()

    total = predicted.sum() + ground_truth.sum()
```

```
    union = np.logical_or(
        predicted,
        ground_truth
    ).sum()

    dice = (
        2 * intersection / total
        if total > 0 else 1.0
    )

    iou = (
        intersection / union
        if union > 0 else 1.0
    )

    return dice, iou

def task9(im, stem, ground_truth=None, seed=None):
    gray = to_gray(im)

    h, w = gray.shape

    if seed is None:
        seed = (h // 2, w // 2)

        print(
            "Task 9: Using image centre as the region-growing seed."
        )

        print(
            "For meaningful analysis, choose a seed inside the target region."
        )

    grow = region_growing(im, seed, 18)
```

```
split_mask, split_visual = split_merge_demo(im)

linked, boundary = edge_linking(im)

outputs = {
    "region_growing": grow,
    "split_merge_residual": split_mask,
    "split_merge_visual": split_visual,
    "edge_linked": linked,
    "edge_boundaries": boundary
}

for name, image in outputs.items():
    save(f"{stem}_T09_{name}.png", image)

if ground_truth is not None:
    rows = []

    for name, output in outputs.items():
        d, i = dice_iou(output, ground_truth)

        rows.append({
            "method": name,
            "Dice": round(d, 4),
            "IoU": round(i, 4)
        })

    write_csv(
        f"{stem}_T09_groundtruth_metrics.csv",
        rows
    )

else:
    print(
        "No annotated ground truth: Dice and IoU are not calculated."
    )
```

```
    show(
        [("Original", im)] + list(outputs.items()),
        f"{stem} - Task 9",
        2
    )

    return outputs

# ============================================================
# TASK 10: COLOUR AND HIGH-RESOLUTION STRATEGIES
# ============================================================

def task10(im, stem):
    h, w = im.shape[:2]

    lab = cv2.cvtColor(
        im,
        cv2.COLOR_BGR2LAB
    )

    L, A, B = cv2.split(lab)

    clahe = cv2.createCLAHE(
        clipLimit=2.0,
        tileGridSize=(8, 8)
    )

    # A: Retain original size and enhance luminance
    retain_L = clahe.apply(L)

    retain = cv2.cvtColor(
        cv2.merge((retain_L, A, B)),
        cv2.COLOR_LAB2BGR
    )
```

```
# B: Resize large image and enhance luminance
scale = min(1.0, 2000.0 / max(h, w))

if scale < 1:
    resized = cv2.resize(
        im,
        (
            max(1, int(w * scale)),
            max(1, int(h * scale))
        ),
        interpolation=cv2.INTER_AREA
    )
else:
    resized = im.copy()

lab2 = cv2.cvtColor(
    resized,
    cv2.COLOR_BGR2LAB
)

l2, a2, b2 = cv2.split(lab2)

l2 = clahe.apply(l2)

modified = cv2.cvtColor(
    cv2.merge((l2, a2, b2)),
    cv2.COLOR_LAB2BGR
)

# C: Candidate damage mask and inpainting
gray = to_gray(im)

local = cv2.medianBlur(gray, 9)

residual = cv2.absdiff(
    gray,
```

```
    local
)

mask = cv2.threshold(
    residual,
    28,
    255,
    cv2.THRESH_BINARY
)[1]

kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE,
    (3, 3)
)

mask = cv2.morphologyEx(
    mask,
    cv2.MORPH_OPEN,
    kernel
)

corrected = cv2.cvtColor(
    cv2.merge((
        clahe.apply(L),
        A,
        B
    )),
    cv2.COLOR_LAB2BGR
)

redesigned = cv2.inpaint(
    corrected,
    mask,
    3,
    cv2.INPAINT_TELEA
)
```

```
    outputs = {
        "A_retain": retain,
        "B_modify_resize_luminance": modified,
        "C_redesign_inpaint": redesigned,
        "C_candidate_damage_mask": mask
    }

    for name, image in outputs.items():
        save(f"{stem}_T10_{name}.png", image)

    print(
        "Task 10: Inspect the candidate damage mask before accepting inpainting."
    )

    print(
        "The mask may select genuine texture, edges or fine photograph details."
    )

    show(
        [("Original", im)] + list(outputs.items()),
        f"{stem} - Task 10",
        2
    )

    return outputs

# ============================================================
# MAIN EXPERIMENT RUNNER
# ============================================================

def run_one(path):
    image = load_image(path)

    stem = Path(path).stem.replace(" ", "_")
```

```
save(f"{stem}_original.png", image)

print("\n========================================")
print("OLD PHOTOGRAPH RESTORATION")
print("========================================")

print("Image:", path)

print(
    "Dimensions:",
    image.shape[1],
    "x",
    image.shape[0]
)

tasks = [
    ("Task 1 - Grey-level transformations", task1),
    ("Task 2 - Histogram enhancement", task2),
    ("Task 3 - Spatial filters", task3),
    ("Task 4 - Sharpening", task4),
    ("Task 5 - Frequency low-pass", task5),
    ("Task 6 - High-pass and homomorphic", task6),
    ("Task 7 - Point, line and edge detection", task7),
    ("Task 8 - Thresholding", task8)
]

timing_rows = []

for label, function in tasks:
    print("\nRunning:", label)

    _, seconds = timed(
        function,
        image,
        stem
    )
```

```
    print(
        label,
        "completed in",
        round(seconds, 3),
        "seconds"
    )

    timing_rows.append({
        "task": label,
        "time_seconds": round(seconds, 3)
    })

# Optional ground-truth mask for Task 9
ground_truth = None

if COLAB:
    answer = input(
        "\nUpload a ground-truth damage mask for Task 9? (y/n): "
    ).strip().lower()

    if answer == "y":
        print(
            "Upload a binary mask image. White represents the target region."
        )

        uploaded = files.upload()

        for filename in uploaded:
            if Path(filename).suffix.lower() in FORMATS:
                ground_truth = cv2.imread(
                    filename,
                    cv2.IMREAD_GRAYSCALE
                )

                if ground_truth is not None:
```

```
                ground_truth = cv2.resize(
                    ground_truth,
                    (image.shape[1], image.shape[0]),
                    interpolation=cv2.INTER_NEAREST
                )

                _, ground_truth = cv2.threshold(
                    ground_truth,
                    127,
                    255,
                    cv2.THRESH_BINARY
                )

                break

print("\nRunning: Task 9 - Image segmentation")

_, seconds = timed(
    task9,
    image,
    stem,
    ground_truth
)

print(
    "Task 9 completed in",
    round(seconds, 3),
    "seconds"
)

timing_rows.append({
    "task": "Task 9 - Image segmentation",
    "time_seconds": round(seconds, 3)
})

print("\nRunning: Task 10 - Colour and high-resolution strategies")
```

```
    _, seconds = timed(
        task10,
        image,
        stem
    )

    print(
        "Task 10 completed in",
        round(seconds, 3),
        "seconds"
    )

    timing_rows.append({
        "task": "Task 10 - Colour and high-resolution strategies",
        "time_seconds": round(seconds, 3)
    })

    write_csv(
        f"{stem}_execution_times.csv",
        timing_rows
    )

    print("\nAll tasks completed for:", path)

# ============================================================
# MAIN FUNCTION
# ============================================================

def main():
    if COLAB:
        print("Upload your old photograph image(s).")

        uploaded = files.upload()

        paths = [
```

```
        name for name in uploaded
        if Path(name).suffix.lower() in FORMATS
    ]

else:
    import argparse

    parser = argparse.ArgumentParser(
        description="Old Photograph Restoration"
    )

    parser.add_argument(
        "images",
        nargs="+",
        help="Image file paths"
    )

    args = parser.parse_args()

    paths = args.images

if not paths:
    raise SystemExit(
        "No supported images were uploaded or supplied."
    )

for path in paths:
    run_one(path)

print("\n========================================")
print("PROJECT EXECUTION COMPLETED")
print("========================================")

print("Output folder:", OUT.resolve())

if COLAB:
```

```
    archive = shutil.make_archive(
        "/content/old_photo_restoration_results",
        "zip",
        root_dir=str(OUT)
    )

    print("Created results archive:", archive)

    files.download(archive)

# ============================================================
# PROGRAM START
# ============================================================

if __name__ == "__main__":
    main()
```

OUTPUT:





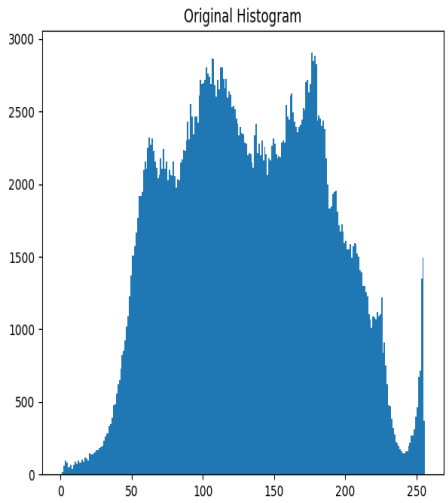


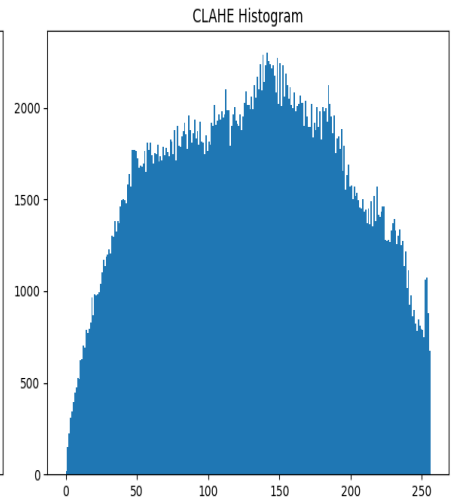




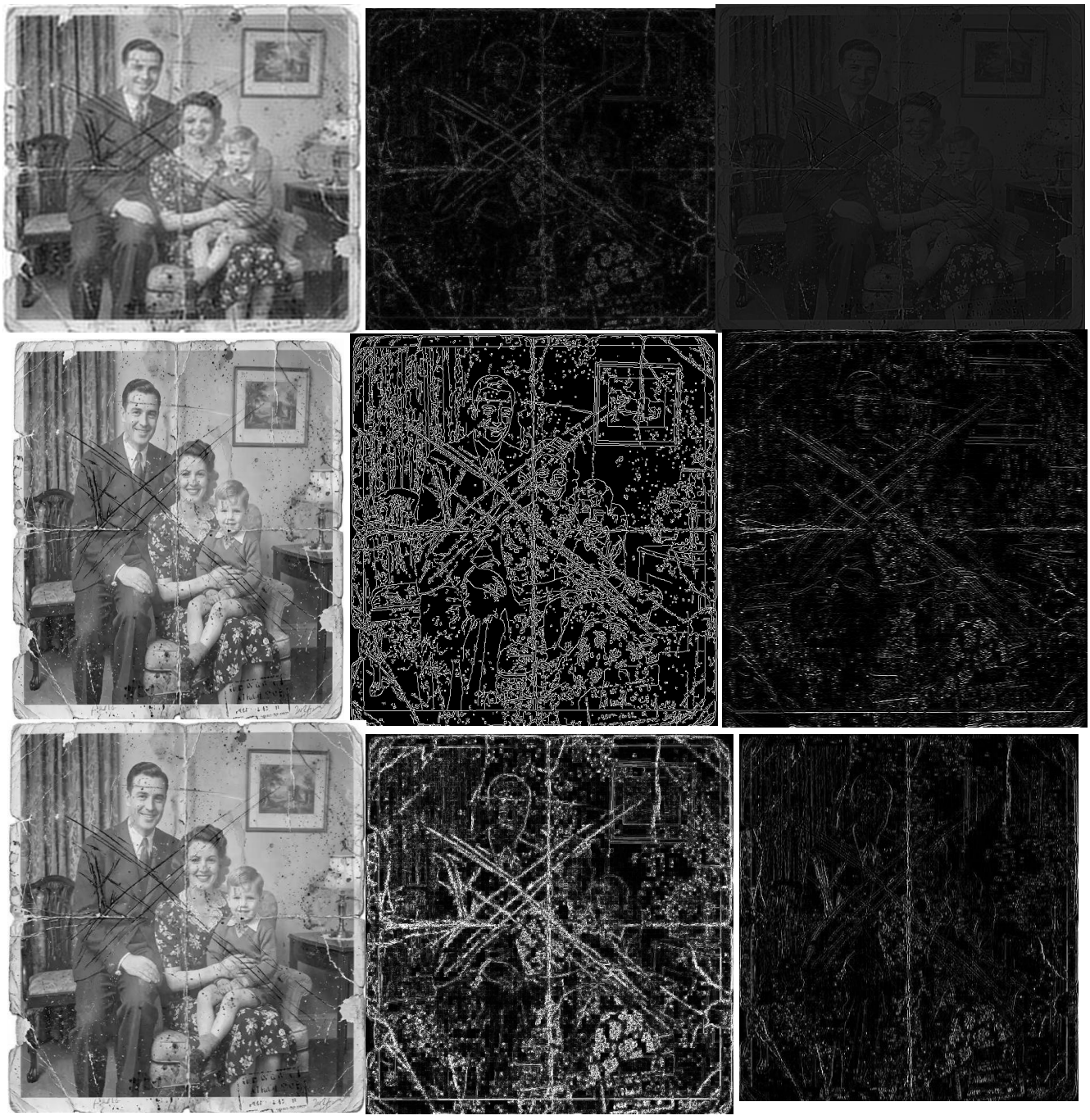


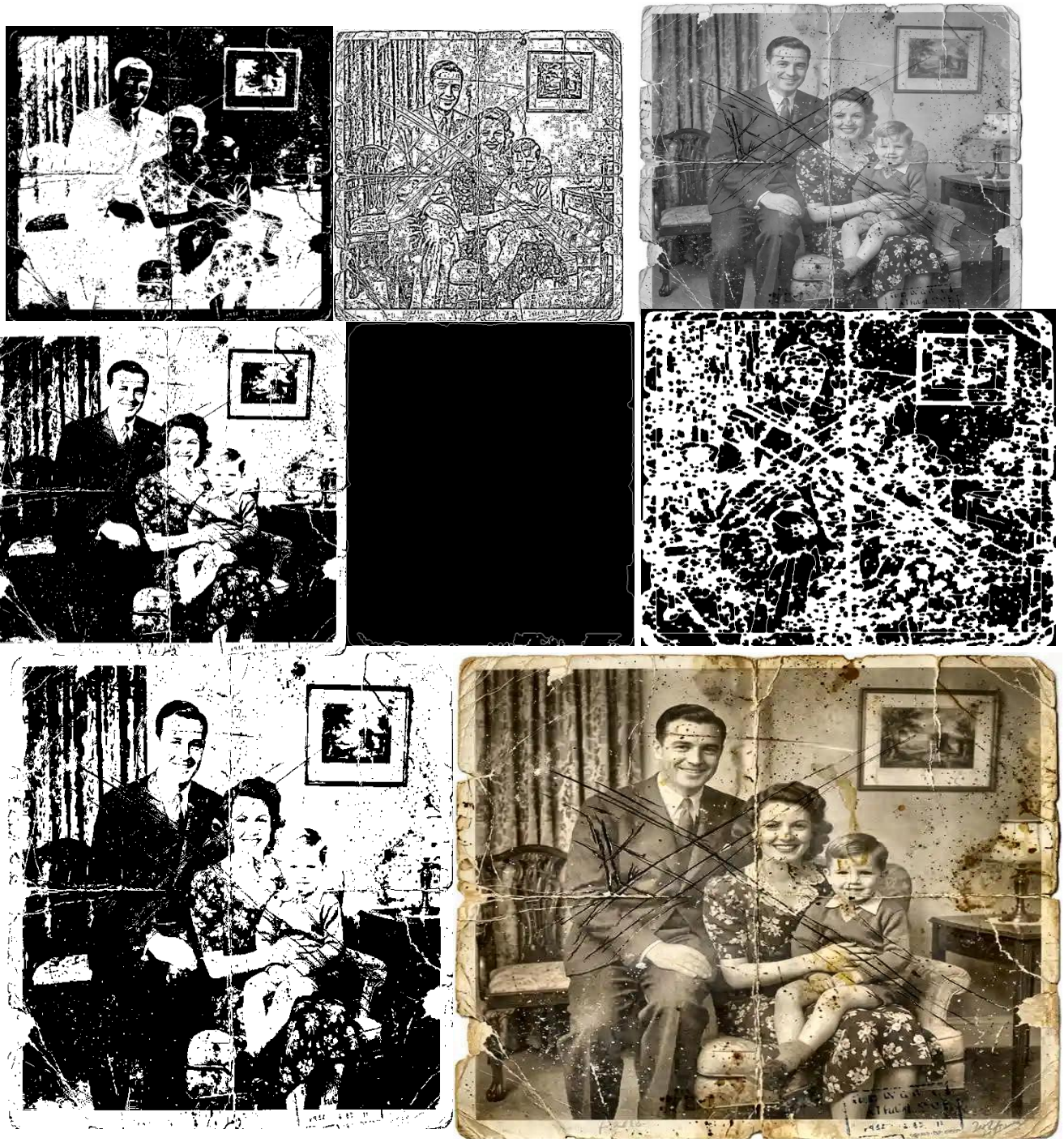



Original Histogram
3000
2500
2000
1500
1000
500
0
0
50
100
150
200
250


CLAHE Histogram
2000
1500
1000
500
0
0
50
100
150
200
250







ORIGINAL IMAGE:



RESTORED IMAGE:



OVERALL PROJECT RESULT

The Old Photograph Restoration project was implemented successfully using Python, OpenCV, NumPy and Matplotlib.

Ten different image-processing tasks were performed on uploaded digital photographs. Grey-level transformations modified pixel intensity values, while histogram enhancement improved image contrast.

Spatial filtering demonstrated different approaches to noise reduction. Image sharpening improved the visibility of certain edges and details.

Frequency-domain low-pass and high-pass filtering were performed using Fourier transformation. Homomorphic filtering demonstrated the processing of illumination-related variations.

Point, line and edge detection were used to analyse image structures. Different thresholding techniques were implemented to separate pixels based on intensity values.

Image segmentation was demonstrated using region growing, recursive split-based processing and edge linking. Finally, colour enhancement and inpainting were used to demonstrate luminance adjustment and estimation of selected damaged regions.

The program also saved output images and recorded the execution time of the processing tasks. Optional Dice and IoU measurements were included for evaluating segmentation when a manually annotated ground-truth mask was provided.

The project demonstrates that digital image-processing techniques can improve the visual appearance of old photographs and help analyse damaged regions. However, the results depend on the input image, processing parameters and quality of the selected damage mask.

ADVANTAGES OF THE PROJECT

1. Uses Python and OpenCV for digital photograph processing.
2. Supports common image formats.
3. Implements ten different image-processing tasks.
4. Improves contrast using histogram enhancement.
5. Demonstrates noise reduction through spatial filters.
6. Enhances image edges using sharpening techniques.
7. Supports frequency-domain image processing.
8. Performs edge detection and thresholding.
9. Demonstrates different image segmentation approaches.
10. Uses inpainting to estimate selected damaged regions.
11. Saves processed images automatically.
12. Records processing-time information.

LIMITATIONS OF THE PROJECT

1. The system cannot recover all missing original image details.
2. Automatic damage detection may identify genuine image features as damage.
3. Incorrect threshold values may produce poor results.
4. Excessive filtering may remove important details.
5. Sharpening may increase visible noise.

6. Inpainting may produce estimated details rather than historically accurate information.
7. Colour enhancement does not establish the original historical colours.
8. Processing large photographs may require more memory and time.
9. Region growing depends on the selected seed point.
10. Segmentation quality is difficult to measure without annotated ground-truth images.

FUTURE ENHANCEMENTS

The project can be improved by including advanced image-restoration algorithms and deep learning techniques.

Artificial intelligence models can be used to identify scratches, stains and damaged regions more accurately. Deep learning-based image restoration may improve the handling of complex damage.

Automatic seed selection can be added to the segmentation module. Better damage masks can be created using manual annotation or trained detection models.

The system can also be developed into a graphical user interface that allows users to upload photographs, select restoration techniques and download enhanced images.

Additional features such as face restoration, colour correction, image super-resolution and batch processing can be included in future versions.

CONCLUSION

Old Photograph Restoration Using Image Processing Techniques was successfully developed as a software-based project using Python and OpenCV.

The project demonstrates ten different image-processing tasks that can be applied to old photographs. These include grey-level transformations, histogram enhancement, spatial filtering, sharpening, frequency-domain filtering, edge detection, thresholding, segmentation, colour enhancement and inpainting.

The implementation helps understand the working principles of digital image processing and provides practical experience with image-processing libraries.

The output images demonstrate different enhancement and analysis methods. The project also shows the importance of selecting suitable parameters and inspecting processed results before accepting them as restored photographs.

Overall, this project provides a useful foundation for understanding digital photograph restoration and can be extended using advanced image-processing and artificial intelligence techniques.

REFERENCES

1. R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th Edition, Pearson, 2018.
2. G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly Media, 2008.
3. OpenCV Documentation, Image Processing in OpenCV, https://docs.opencv.org/
4. NumPy Documentation, https://numpy.org/doc/
5. Matplotlib Documentation, https://matplotlib.org/stable/
6. Google Colab Documentation, https://colab.research.google.com/
7. M. Sonka, V. Hlavac and R. Boyle, Image Processing, Analysis, and Machine Vision, Cengage Learning.
8. P. Soille, Morphological Image Analysis: Principles and Applications, Springer, 2003.